

docs_chain

Ешқандай бақыланатын құжат үйлесімсіздікке түсе алмайды — мазмұндық хэштеу, екі бағытты, атомдық.

Қысқаша мазмұны

Docs Chain — әмбебап, Go-ті қолданған екі бағытты құжат-және-дерекқор тәуелділік тарату қозғалтқышы. Тіркелген тізбектегі кез келген мүше өзгерген кезде — Markdown бастапқы мәтіні, HTML/PDF/DOCX экспорты немесе SQLite дерекқоры — ол өзгерісті мазмұндық хэш арқылы анықтап, барлық байланысқан мүшелерге атомдық түрде таратады.

Қысқаша сипаттама

Құжаттар мен дерекқорларды үйлесімді сақтайтын Go қозғалтқышы. Salsa стиліндегі мазмұндық хэштеу арқылы өсімдік қайта есептеуді DAG (Кан топологиялық реттеу, ерте тоқтату, екі бағытты синхрондау қырлары, атомдық атау ауыстыру + SQLite транзакциялық коммиттер) бойынша жүзеге асырады, байланысқан кез келген артефакт өзгергенде экспорттарды қайта жаңартады.

Кеңейтілген сипаттама

Docs Chain — бұл бір рет артық «Markdown өзгергенде PDF қайта жаңарту» деген нәзік shell скриптін жазғаннан кейін жасалатын нәрсе. Ол қолдан жасалған синхрондау байланыстардың бәрін нағыз қозғалтқышпен алмастырады. Жобаның құжаттары мен дерекқорларын тізбек мүшелері ретінде модельдеп, кез келген мүше өзгергенде, ол өзгерісті барлық байланысқан мүшелерге әр жарияланған бағытта таратып, қайта жаңарту мен экспорттауды атомдық түрде орындайды, сондықтан ешқандай бақыланатын артефакт үйлесімсіздікке түсе алмайды. Жобалау принциптері инкременталды құрылыс жүйелерінен алынған, скрипттерден емес: өзгеріс анықтауы **мазмұндық хэш арқылы, уақыт белгісі арқылы емес**, сондықтан touch ештеңеге әсер

етпейді, ал бір байттық өзгеріс дәл қажетті қайта құруларды ғана іске қосады — жалған алармдар мен өткізілген өзгерістер жоқ. Бір жолда формальды түрде айтқанда, бұл Salsa стиліндегі мазмұндық хэштеу арқылы өсімдік қайта есептеу DAG бойынша, Кан топологиялық реттеумен, өзгермеген тармақтарды ерте кесумен, жарияланған билік екі бағытты sync қырларымен, атомдық атау ауыстыру мен SQLite транзакциялық коммиттерімен жүзеге асырылады, сондықтан тарату барысында апат болса, жартылай жазылған экспорт қалмайды. Ол vasic-digital субмодулі ретінде жеткізіліп, HelixConstitution субмодулінің негізгі бөлігі ретінде қолданылады, сондықтан конституцияны қабылдаған кез келген жоба Docs Chain-ды дайын күйінде алып, өз тізбектерін контекстік YAML арқылы тіркеуі мүмкін. Іске асыру статусы ашық көрсетілген (конституцияның §11.4.6 бөлімі бойынша): 1–4 кезеңдері (негізгі DAG + хэштеу, түйін адаптерлері/түрлендірулері, атомдықтығы бар тарату оркестраторы, конфигурацияға негізделген көп контексті CLI sync/verify/doctor/graph/watch мүмкіндіктерімен) іске асырылып, сынақтан өткен; 4b кезеңі жалпы екі бағытты md-to-sqlite/sqlite-to-md кіріктірілген модульдерін (таза Go, жол деңгейіндегі үйлесімсіздік, байт тұрақты айналым) және colorize-html кіріктірілген модулін қосады; 5 кезең толық нақты екі нүкте арасындағы сынақтар іске асырылып, ӨТТІ. 6–7 кезеңдері (конституция таратуы, ATMOSphere байланысы) ЖОСПАРЛАНҒАН және операторлық бақылауға алынған. Herald — бірінші нақты төменгі тұтынушы, 66 құжаттық көп форматты корпус синхрондап, тазалықты тексереді.

Неліктен жасадық

Құжаттама, экспорттар және дерекқорлар қолмен немесе нәзік скрипттер арқылы қолдау кезінде бір-бірінен алшақтайды. Docs Chain синхрондауді механикалық, мазмұндық хэш дәлдігімен және атомдық түрде жүзеге асырады, сондықтан тізбектегі кез келген өзгеріс төменгі (және жоғарғы) ағымдағы барлық нәрсені дұрыс және қауіпсіз жаңартады.

Мазмұн

Неге бұл ойынды өзгертетін шешім

Компиляторлар мен құрылымдық жүйелердің авторлары әдеттегідей қабылдайтын, қиындықпен жеңілген дұрыстық кепілдіктері – мазмұндық хештелген тәуелділік графтары, минималды қайта есептеу, атомдық коммиттер – осылардың бәрі енді құжаттама мен дерекқорларға бағытталған. Бұл сала тарихи тұрғыда сгон жұмыстары мен жақсы ниеттерге

сүйенген. Шын мәніндегі екі бағытты синхрондау дегеніміз – бастапқы дерек көзі мен оның экспорты арасындағы байланыстың екі жаққа да қатаң сақталатынын білдіреді. Сондықтан “құжаттама ескірген” немесе “экспорт бастапқы дерек көзімен сәйкес келмейді” деген мәселелер қайталанып отыратын қателер емес, жүйенің мүмкін етпейтін күйлеріне айналады.

Инновациялық жақтары

- Мазмұндық хеш (уақыт белгісі емес) бойынша DAG-тағы инкременталды қайта есептеу және ерте тоқтату.
- Екі бағытты, анықталған билік синхрондау шеттері (құжаттама ↔ экспорт ↔ SQLite).
- Атомдық атау ауыстыру + SQLite транзакциялық коммит апатсыз тарату үшін.
- Таза-Go md-to-sqlite/sqlite-to-md айналымы және жол деңгейіндегі ауытқуды анықтау.

Мәселелер мен шешімдер

- **Жалған қайта құрулар:** уақыт белгілерінің орнына мазмұндық хешті анықтау арқылы шешілді.
- **Жартылай/бұзылған жаңартулар:** атомдық атау ауыстыру және SQLite транзакциялары арқылы шешілді.
- **Көп мүшелі реттіліктің дұрыстығы:** Канның топологиялық реттеуі + ерте тоқтату арқылы шешілді.
- **Шындыққа сай мүмкіндіктерді хабарлау:** әр кезеңді §11.4.6 бөліміне сәйкес IMPLEMENTED немесе PLANNED деп белгілеу арқылы шешілді.

Технологиялық стек (неге және қалай)

- **Go** — бүкіл қозғалтқыш (internal/hash, graph, adapter, orchestrator, config, state, runner, cmd/docs_chain).
- **DAG + Канның топологиялық сорттауы** — тәуелділіктерді реттеу және ерте тоқтату.
- **SQLite (таза-Go modernc)** — дерекқор мүшелері мен транзакциялық коммиттер.
- **fsnotify** — тікелей тарату үшін watch демоны.
- **YAML конфигурациясы** — әр контекстегі тізбектерді тіркеу.
- **exes: түрлендірулер** — қосылатын Markdown→HTML/PDF/DOCX генерациясы.

Жол картасының шындығы: 6-7 кезеңдер (конституция тарату, ATMOSphere байланысы) PLANNED / операторлық шектеулі - әзірге жеткізілмеген.